SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Solutions:

1. ATM Withdrawal Module – Black Box Testing

The ATM withdrawal module can be reverse engineered by observing its inputs, outputs, and error messages. The system can be divided into PIN validation, balance verification, withdrawal limit checking, cash dispensing, and receipt generation. Equivalence Class Partitioning (ECP) divides inputs into valid and invalid groups, such as valid and invalid PINs, sufficient and insufficient balances, and permitted and prohibited withdrawal amounts.

Boundary Value Analysis (BVA) tests values near the minimum and maximum withdrawal limits. For example, if the withdrawal limit is ₹500 to ₹25,000, test ₹499, ₹500, ₹501, ₹24,999, ₹25,000, and ₹25,001. These test cases help identify errors at the limits. Other important cases include incorrect PINs, insufficient funds, daily limit violations, and network failures.

The existing testing is inadequate if it covers only successful withdrawals and normal inputs. An improved test suite should cover all valid and invalid input classes, boundary values, error messages, and transaction failures. Expected results should be compared with actual outputs to identify defects. Combining ECP and BVA provides better coverage and improves the reliability and security of the ATM module.

2. White Box Testing – Control Flow Graph and Path Testing

White box testing can be reverse engineered by examining source code snippets and execution traces. A Control Flow Graph (CFG) represents program execution using nodes for statements and edges for control flow. Decision statements such as if-else and switch, along with loops such as for and while, must be identified. Each decision creates alternative execution paths that need to be tested.

The existing test adequacy can be evaluated using statement coverage, branch coverage, and path coverage. Statement coverage checks whether every statement executes, while branch coverage checks whether every

decision outcome is executed. Path coverage checks different execution paths through the program. For example, a withdrawal module may have paths for valid PIN, invalid PIN, sufficient balance, and insufficient balance. Achieving 100% statement coverage alone does not guarantee that all decision outcomes have been tested.

An improved path testing strategy should identify independent execution paths using the CFG and design test cases for each path. Tests should cover both true and false outcomes of every decision, loop execution with zero, one, and multiple iterations, and combinations of conditions. Cyclomatic complexity can help estimate the number of independent paths that should be tested. This approach improves code coverage and helps detect logical errors, incorrect conditions, and unexpected program behavior.

3. Web Application Login and Transaction System – Combined Testing

The web application's testing approach can be reverse engineered by analyzing defect logs and test results. The system can be divided into login authentication, session management, input validation, transaction processing, and database updates. Defects involving invalid credentials or incorrect error messages may indicate black box testing, while defects involving unexecuted branches or incorrect internal logic may indicate weaknesses in white box testing.

Equivalence Class Partitioning and Boundary Value Analysis can be used to test login fields and transaction amounts. For example, login inputs can be divided into valid credentials, invalid credentials, empty fields, and incorrectly formatted inputs. If the transaction limit is ₹100 to ₹50,000, boundary tests should include ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001. Path testing should cover successful login, failed login, session expiration, successful transactions, insufficient balance, and transaction rejection.

The reconstructed testing strategy should combine black box and white box techniques. Black box testing verifies that the application produces correct outputs for valid and invalid inputs, while white box testing checks internal branches and execution paths. Additional cases should cover repeated login failures, unauthorized transactions, duplicate payments, database errors, and session timeouts. This integrated approach improves functional correctness, security, and transaction reliability while reducing the risk of defects in production.

4. Legacy Inventory Management System – Black Box Testing

The inventory management system can be reverse engineered by studying successful and failed stock update transactions and low-stock alerts. The system can be divided into stock addition, stock reduction, inventory validation, database updating, and alert generation. Equivalence Class Partitioning can identify valid stock quantities, zero quantities, negative quantities, and quantities exceeding permitted limits. Stock levels can also be grouped into normal, low-stock, and out-of-stock categories.

Boundary Value Analysis helps identify hidden conditions in stock updates and alert generation. For example, if the low-stock threshold is 10 units, test stock levels of 9, 10, and 11 units to determine when an alert is generated. Stock updates should also be tested with zero, negative, and unusually large quantities. Transaction logs can reveal whether the system incorrectly permits negative inventory, fails to generate alerts, or updates stock values inaccurately.

An improved black box test suite should include valid stock additions, valid stock reductions, insufficient stock, duplicate updates, invalid quantities, and simultaneous update requests. Expected stock values and alert messages must be compared with actual results. The tests should verify that inventory never becomes negative unless explicitly permitted by the requirements and that alerts occur at the correct threshold. This approach improves inventory accuracy and prevents stock inconsistencies.

5. Library Management Module – White Box Testing Using Execution Profiles

The compiled library management module can be analyzed using CPU and memory execution profiles collected during stress testing. High CPU usage may indicate repeated loops, expensive searches, or inefficient sorting operations, while memory spikes may indicate large data structures, repeated allocations, or memory leaks. A conceptual Control Flow Graph can be constructed by identifying likely processing stages, such as book searching, member validation, borrowing, returning, and fine calculation. However, execution profiles alone cannot reveal the exact source code or prove the existence of a particular branch.

The path testing strategy should focus on resource-intensive operations and important decision points. For example, book searching should be tested with an empty database, a small collection, and a large collection. Borrowing operations should cover available books, unavailable books, invalid member details, and borrowing-limit violations. Loops should be examined for zero, one, and many iterations, while repeated transactions can reveal performance degradation and memory problems.

The reconstructed strategy should combine path testing with stress, load, and endurance testing. Resource usage should be monitored while executing important paths under increasing workloads. Tests should also check response time, memory consumption, and system stability during repeated borrowing and returning operations. This approach helps identify inefficient paths, resource exhaustion, and potential failures, improving the overall stability and performance of the library management system.

6. Mobile Payment Gateway – State Transition Testing

The mobile payment gateway can be reverse engineered by examining API responses for Pending, Success, Reversal, and Timeout states. The system can be divided into payment initiation, payment processing, confirmation, timeout handling, and reversal processing. State Transition Testing is the likely technique used to analyze the system because the validity of an operation depends on the current transaction state and the event that triggers the next state.

A state transition model can represent the legal transitions. For example, Pending may transition to Success, Reversal, or Timeout, depending on the payment processor's rules. A timed-out transaction may later receive a delayed confirmation, so the system must define how such cases are handled. Illegal transitions, such as processing a completed payment as a new payment without authorization, should also be tested. Each test should verify the resulting state, API response, and transaction record.

An improved test suite should cover every legal transition and every important illegal transition. Tests should include successful payments, failed payments, timeouts, reversal requests, duplicate callbacks, delayed responses, and repeated reversal attempts. The system must also prevent duplicate charges and

ensure that transaction states remain consistent between the application and payment processor. This approach improves payment accuracy, fault handling, and transaction security.

7. Encrypted Communication Module – Fuzzing and Boundary Value Analysis

The encrypted communication module can be reverse engineered by supplying different input values and observing encrypted outputs, validation errors, and system behavior. The module can be divided into input validation, data formatting, encryption, and error handling. Fuzzing involves testing unexpected, malformed, oversized, and randomly generated inputs to discover weaknesses. Boundary Value Analysis can identify failures caused by input lengths near the minimum and maximum permitted sizes.

The test suite should include empty inputs, valid inputs, maximum-length inputs, oversized messages, invalid characters, malformed encodings, and missing parameters. For example, if the maximum permitted message length is 1,024 bytes, test lengths of 1,023, 1,024, and 1,025 bytes. The module should reject invalid inputs safely and avoid crashes, memory corruption, or unexpected output. Special characters and malformed data should also be tested to identify possible injection-related weaknesses.

An improved strategy should combine fuzz testing, boundary testing, negative testing, and security testing. Each test should verify that valid data is processed correctly and invalid data is rejected without exposing sensitive information. Input lengths must be validated before processing, and secure memory handling should be verified through suitable tools. These techniques help identify buffer overflow risks, input validation defects, and error-handling weaknesses, improving the security and reliability of the communication module.

8. Insurance Premium Calculator – White Box Testing

The insurance premium calculator achieves 100% statement coverage but still misses defects in multi-condition if statements. This suggests that the original testing approach focused on executing every statement rather than verifying every decision outcome or condition. For example, a premium calculation may depend on age, policy type, and driving history. Executing the calculation statement once does not guarantee that all combinations of these conditions have been tested.

An improved Control Flow Graph should identify every decision node and its possible branches. Branch coverage requires both true and false outcomes of each decision to execute. Condition coverage checks whether each individual Boolean condition becomes true and false. Modified Condition/Decision Coverage (MC/DC) can further verify that each condition independently affects the overall decision result. For example, tests should cover eligible and ineligible ages, different policy types, and positive or negative risk factors.

The reconstructed test strategy should include independent paths through the premium calculation logic, with test data selected to exercise each decision and condition. Cyclomatic complexity can estimate the number of independent paths in the CFG, although complete path coverage may be impractical when loops create many possible paths. Combining branch coverage, condition coverage, and MC/DC where

appropriate provides stronger assurance than statement coverage alone. This reduces the risk of incorrect premium calculations and improves the accuracy of policy decisions.

9. Cloud-Based File Synchronization Tool – Equivalence Partitioning and Stress Testing

The file synchronization tool can be reverse engineered by analyzing user activity logs and reports of lost files. The system can be divided into file scanning, change detection, uploading, downloading, conflict resolution, and synchronization confirmation. Equivalence Class Partitioning can group files by size, type, and synchronization status. For example, files may belong to valid or unsupported formats, small or large sizes, and unchanged or modified states. Network conditions can also be classified as stable, slow, intermittent, or disconnected.

Boundary Value Analysis should test file sizes near the supported minimum and maximum limits. If the system supports files up to 1 GB, test files just below, exactly at, and just above that limit. Additional tests should examine empty files, large files, duplicate filenames, simultaneous edits, interrupted uploads, and insufficient storage. Stress testing should evaluate the synchronization process when many files are transferred concurrently or network connectivity repeatedly fails.

The improved testing strategy should combine ECP, BVA, stress testing, and recovery testing. Every upload or download should be checked for data integrity using suitable file comparisons or checksums. Interrupted operations should resume safely without losing data or creating inconsistent copies. Conflict resolution must also be tested when the same file is modified on multiple devices. These tests improve synchronization reliability and reduce the risk of lost, corrupted, or duplicated files.

10. E-Commerce Search and Filter Module – Decision Table Testing

The e-commerce search and filter module can be reverse engineered by examining SQL queries, search parameters, and returned datasets. The system can be divided into keyword search, category filtering, price filtering, availability checking, sorting, and result pagination. The generated SQL queries can reveal whether conditions are combined using AND, OR, and NOT. Comparing query results with expected datasets helps identify the hidden decision-making logic and possible filtering defects.

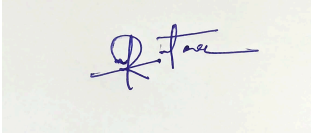
Decision Table Testing is suitable because the results depend on combinations of conditions. For example, a customer may search for a product by keyword, select a category, specify a price range, and require the product to be in stock. Test cases should cover situations where all conditions match, only some conditions match, no conditions match, or conflicting filters are supplied. Boundary tests should also cover minimum and maximum prices, empty search terms, and pages with no results.

An improved strategy should construct a decision table containing relevant conditions and expected actions for each meaningful combination. Tests should verify that AND returns products satisfying all required conditions, OR includes products satisfying at least one alternative, and NOT excludes the intended products. The generated SQL queries should also be checked for correct parameter handling and protection against SQL injection. This approach improves search accuracy, filter completeness, and database security while helping ensure that users receive the expected results.

Code of Conduct:

I Prartana T(192421099) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation